

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

ПРОТОКОЛ
лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»

Виконав(-ла):	Студент групи AI-242
	Левентій Дмитро Олександрович
Перевірів:	ас. Баранов О.М.

Одеса – 2026

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку — просте фактичне питання, потім — задача на міркування (5 запусків для кожної моделі).

Таблиця 1 — Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	223	5
Час, с	7,64	0,41
Токенів/с	31,2	36,9

Таблиця 2 — Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1320,2	203,6
Час, с	45,17	6,15

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Токени пішли в поле `thinking` — модель прогнала внутрішні роздуми над очевидним фактом, перш ніж дати ту саму відповідь.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання не виправдане?

Відповідь: Перевага — на задачах у кілька кроків (математика, логіка, планування), де проміжний результат потрібно тримати в тексті. На простих фактичних питаннях і коротких відповідях режим міркування лише витрачає час і токени.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Міркувальну модель додатково донавчають (напр. RL) генерувати ланцюжок роздумів (`chain-of-thought`) перед фінальною відповіддю. Інструктивну вчать одразу видавати готову відповідь без проміжного тексту.

4. Чи означає більша кількість згенерованих tokenів у міркувальній моделі, що вона «розумніша» за інструктивну?

Відповідь: Ні. Кількість tokenів характеризує обсяг роботи, яку модель собі призначила, а не якість чи «розум» — вона може довго міркувати і все одно помилитися.

5. Як пояснити різницю у швидкості генерації (tokenів за секунду) між моделями `qwen3:4b` та `llama3.2:3b`?

Відповідь: Tokenів/с у моделей приблизно однакові (31,2 і 36,9) — це показник заліза й розміру моделі. Різниця в загальному часі відповіді пояснюється кількістю згенерованих tokenів (223 проти 5), а не швидкістю обладнання.

Висновок за завданням 1

Експеримент показав, що два показники — швидкість генерації та загальний час відповіді — не можна плутати. Перший характеризує обладнання й розмір моделі, другий — обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 40 разів. На задачі про ручку й зошит ситуація протилежна — саме розписаний хід дозволяє уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі варто обирати під тип задачі.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Варіант: «Поясни різницю між списком і кортежем у Python.» Виконано для двох моделей із двома варіантами system-повідомлення.

Таблиця 3 — Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	К-сть речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	~230	українською	4	так (коротко й по суті)
llama3.2:3b	укр., 2 речення	~330	українською	2	так
qwen3:4b	лаконічний асистент	212	українською	2	так
qwen3:4b	укр., 2 речення	~330	українською	2	так

Таблиця 4 — Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	55	1889	1944
llama3.2:3b	укр., 2 речення	63	99	162
qwen3:4b	лаконічний асистент	65	2445	2510
qwen3:4b	укр., 2 речення	74	644	718

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Формально обидві моделі вклались приблизно в однаковий видимий обсяг (~330 символів, 2 речення), проте llama3.2:3b досягла цього значно економніше — 99 completion_tokens проти 644 у qwen3:4b. Тобто саме видиму інструкцію щодо кількості речень точніше й «дешевше» виконала llama3.2:3b.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: У видимому полі response — ні, обидві моделі в обох прогонах відповіли українською (табл.

3). Втім, у qwen3:4b неможливо виключити, що приховані роздуми (thinking) велись іншою мовою, оскільки користувачу вони не показуються.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: prompt_tokens зріс незначно — з 55 до 63 у llama3.2:3b і з 65 до 74 у qwen3:4b, тобто приблизно на 8–9 токенів. Це відповідає довжині нового тексту system-повідомлення «Відповідай лише українською, максимум двома реченнями», яке довше за початкове «Ти лаконічний технічний асистент».

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: Різниця пояснюється тим, що qwen3:4b — міркувальна модель: перед фінальною відповіддю вона генерує приховане поле thinking, яке також рахується у completion_tokens, хоча користувач його не бачить. Інструктивна llama3.2:3b одразу видає готову відповідь без цього прихованого етапу.

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Здатність слідувати інструкціям залежить від якості інструктивного донавчання конкретної моделі: менші й простіші моделі гірше узагальнюють вимоги, які виходять за межі типових патернів, побачених під час навчання.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

System-повідомлення зазвичай має вищий пріоритет за задумом донавчання (RLHF/instruction-tuning привчають модель ставити його на перше місце), але це не жорстка гарантія — при конфлікті з сильним патерном із тренувальних даних модель може його проігнорувати.

3. Чому міркувальна модель витрачає більше вихідних tokenів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Тому що частина її completion_tokens іде не в response, а в приховане поле thinking — внутрішній чернетковий хід міркувань, який генерується завжди, навіть для простих запитів.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає? Однозначної гарантії немає: більшість моделей навчені надавати перевагу system, але на практиці нерідко «перемагає» останнє або найбільш конкретне за формулюванням повідомлення, тобто user-запит.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі? Малі моделі мають меншу ємність і навчались на менших обсягах даних, тому рідкісніші патерни — зокрема якісна українська мова — засвоюються ними гірше, ніж великими моделями з ширшим і різноманітнішим корпусом.

Крок 7 (чат-бот): відповіді наведено вище.

Висновок за завданням 2

System-повідомлення дійсно керує поведінкою моделі — зміна лише одного рядка (system) скоротила completion_tokens у llama3.2:3b у 19 разів (з 1889 до 99), а видиму відповідь звела до вимог. Проте його надійність обмежена: воно не усуває приховані витрати tokenів у міркувальних моделях і не гарантує дотримання інструкцій за конфлікту з user-повідомленням. Для критичних сценаріїв (жорсткий ліміт довжини, обов'язковий формат) саму лише інструкцію в system недостатньо — потрібна програмна перевірка (наприклад, підрахунок речень) і повторний запит при порушенні.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Варіант: «Поясни різницю між списком і кортежем у Python. Коротко.» Виконано через providers.py / compare.py.

Таблиця 5 — Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.6-flash
Час відповіді, с	5.40	8.06
completion_tokens	90	297
Довжина відповіді, символів	322	895
Мова відповіді	українською	українською
Фактичних помилок у відповіді	так (нечітке/помилкове пояснення "одночасно для програмістів")	ні

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Змінились лише значення трьох полів у словнику PROVIDERS['cloud'] файлу providers.py (base_url, api_key, model). Логіка make_client() і весь код compare.py залишились без змін — саме в цьому й полягає перевага єдиного OpenAI-сумісного інтерфейсу.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель дала повнішу відповідь — 895 символів проти 322 у локальної. Локальна llama3.2:3b сформулювала нечітке/помилкове пояснення про «одночасне» використання списків і кортежів для програмістів, тоді як хмарна модель коректно й розгорнуто описала різницю у змінюваності (mutable/immutable) з прикладами застосування.

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, зафіксована одна змістова неточність — нечітке формулювання про «одночасне» використання структур, що спотворює суть відмінності. Проблем із самою мовою (граматика, лексика) не виявлено, відповідь була україномовною.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: У нашому вимірюванні хмарна модель відповіла не швидше, а повільніше (8,06 с проти 5,40 с) — очікуваний вииграш у швидкості обчислень не компенсував мережеву затримку та більший обсяг згенерованого тексту (297 tokenів проти 90). Це демонструє, що загальний час відповіді хмарного

провайдера складається з мережевої затримки, черги на сервері й самої генерації, тому «хмара швидша» — не universally правило.

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Він з'явився першим у масовому використанні й виявився простим та достатнім, тому іншим провайдерам було вигідніше підтримати вже звичний розробникам формат, ніж змушувати переписувати клієнтський код під свій власний.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Розмір моделі — важливий, але не єдиний фактор: якість і різноманітність тренувальних даних, інструктивне донавчання й alignment часто впливають на практичну якість відповіді не менше, ніж кількість параметрів.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

У нашому випадку цього не сталося — хмарна відповіла повільніше. У загальному ж випадку, коли хмарна модель швидша, це пояснюється значно потужнішим апаратним забезпеченням (кластери GPU) на боці провайдера, яке з лишком компенсує мережеву затримку, особливо на довгих відповідях.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Дані користувача йдуть на сторонній сервер і потенційно можуть використовуватись для донавчання моделі або зберігатися в логах. У комерційних проєктах це вирішують угодами про нерозголошення й по-retention API, шифруванням каналу, а для найчутливіших даних — переходом на локальні/приватні моделі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Rate limit — це обмеження на кількість запитів або токенів за одиницю часу для безкоштовного акаунта. Розробники обходять це кешуванням однакових запитів, чергами із затримками (backoff), розподілом навантаження між кількома ключами/провайдерами або переходом на платний тариф.

Висновок за завданням 3

Перехід між локальним і хмарним провайдером звівся до зміни кількох рядків конфігурації, що підтверджує практичну цінність єдиного OpenAI-сумісного інтерфейсу. Водночас «хмарна = краща й швидша» — це не аксіома: у нашому вимірюванні хмарна модель відповіла повільніше, хоча й змістовно точніше та повніше. Вибір між ними — це компроміс між якістю відповіді, швидкістю, приватністю даних та залежністю від мережі, а не однозначна перевага одного варіанта.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Виконано через benchmark.py: три повтори з медіаною для кожної комбінації провайдер × промпт.

Таблиця 6 — Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.04	2.418	14.5
локальна	llama3.2:3b	середній	2.074	2.868	78.6
локальна	llama3.2:3b	довгий	2.051	8.68	108.3
хмарна	gemini-3.6-flash	короткий	2.845	2.858	12.0
хмарна	gemini-3.6-flash	середній	6.437	7.318	73.9
хмарна	gemini-3.6-flash	довгий	8.455	16.275	236.6

Таблиця 7 — Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.037	2.04
Всього, с	2.336	2.418

Відповіді на питання (крок 6)

1. У якій моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий (менший) TTFT у локальної моделі — приблизно 2,0–2,1 с проти 2,8–8,5 с у хмарної. Це пояснюється відсутністю мережевої затримки: запит обробляється одразу на тій самій машині, тоді як хмарний запит додатково витрачає час на дорогу пакета через інтернет і чергу на сервері.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — пояснить.

Відповідь: Ні, не збігається. Локальна модель виграє за TTFT на всіх промптах, але на довгому промпті хмарна модель генерує швидше за символами на секунду (236,6 проти 108,3), тобто краще за пропускну здатність генерації завдяки потужнішому апаратному забезпеченню на боці провайдера.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: На короткому промпті локальна модель випереджає хмарну і за TTFT, і за загальним часом. На довгому промпті TTFT хмарної моделі зростає ще сильніше (до 8,5 с), але її швидкість генерації (символів/с) починає суттєво перевищувати локальну, тобто перевага локальної моделі зменшується зі зростанням довжини відповіді.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Розбіжність між повторами невелика (у межах кількох відсотків), однак поодинокий запуск ризикуює потрапити на випадковий викид (мережевий джиттер, фонове навантаження ОС), тому медіана з трьох повторів — виправдана практика для надійнішого результату.

5. Чому холодний старт повільніший за теплий?

Відповідь: При холодному старті модель потрібно спершу завантажити з диска в оперативну пам'ять (load_duration), а при теплому вона вже перебуває в пам'яті й одразу готова обробляти запит, тому додаткового часу на завантаження не витрачається.

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Під час TTFT модель обробляє (кодує) весь вхідний промпт за один прохід і генерує перший вихідний токен; це вимірюють окремо, бо саме цей час користувач бачить як «підвисання» перед появою відповіді, і він залежить від інших факторів (довжини промпту, мережі), ніж швидкість подальшої генерації.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

TTFT хмарної моделі включає фіксовану мережеву затримку туди-й-назад, яку неможливо усунути, тоді як на подальшу генерацію ця затримка вже не впливає так само сильно, а потужне серверне обладнання видає токени швидше за слабший локальний процесор.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Холодний старт — це стан, коли модель ще не завантажена в пам'ять і сервер має зчитати кілька гігабайтів ваг з диска перед першим запитом; наступні запити вже «теплі», бо модель залишається резидентною в пам'яті.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

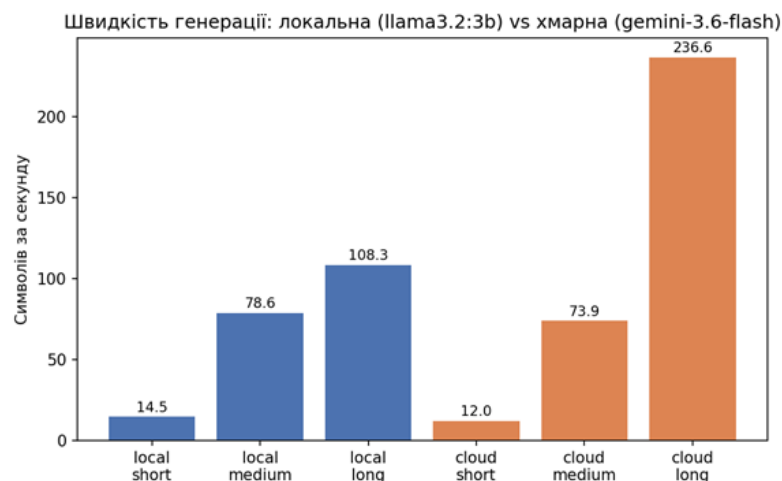
Медіана менш чутлива до випадкових викидів (одноразового затримання мережі, фонового навантаження системи), ніж окреме значення чи навіть середнє арифметичне, тому дає стабільнішу оцінку типової поведінки.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Обробка промпту (prompt evaluation) виконується за один паралельний прохід моделі по всіх вхідних токенах одразу, тоді як генерація відповіді — послідовний авторегресійний процес, де кожен наступний токен обчислюється окремо на основі всіх попередніх.

Висновок за завданням 4

Затримку відповіді не можна характеризувати одним числом: TTFT і швидкість генерації відображають різні аспекти продуктивності і можуть суперечити один одному. У нашому експерименті локальна модель виграє за TTFT на будь-якому промпті, але на довгих відповідях поступається хмарній за пропускну здатність генерації. Для інтерактивного чат-бота критичніший TTFT (користувач очікує швидкої першої реакції), а для фонові обробки великих обсягів тексту важливіша сумарна швидкість генерації символів за секунду.



Завдання 5. Оцінювання якості моделей за рубрикою

Виконано через quality.py: кожне завдання прогнано двічі для кожної моделі, відповіді збережено у quality.json.

Таблиця 13 — Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	1	2	2
5. Логіка	0	2	2
6. Українська	0	2	2
Разом /12	5	12	12

Таблиця 14 — Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	так	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів набрали qwen3:4b і хмарна модель — по 12/12 у обох. llama3.2:3b набрала лише 5/12: основна різниця виникла на завданнях 3 (стиснення тексту), 5 (логіка) і 6 (якість української мови), де вона отримала по 0 балів, а також на завданні 4 (код), де вона отримала лише 1 бал.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Розбіжність між двома прогонами зафіксована лише в llama3.2:3b на завданні 4 (код) — в одному прогоні умову виконано частково, в іншому по-іншому. Для решти моделей і завдань обидва прогони збіглися. Це свідчить, що якість малої моделі менш стабільна від запуску до запуску, тоді як більші моделі поведуться передбачуваніше.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три моделі отримали по 2 бали за завдання 2 — їхні відповіді парсились функцією json.loads() без жодних правок, тобто вимогу «без markdown» дотримано в усіх випадках.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, проблема виявлена у llama3.2:3b (0 балів за завдання 6) — відповідь містила кострубаті конструкції та ознаки калькування з англійської/російської. У qwen3:4b і хмарної моделі якість української мови була природною, обидві отримали максимальний бал.

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Крім рубрик, поширені: людське парне порівняння відповідей (Arena/Elo-рейтинги), автоматизовані бенчмарки (MMLU, GSM8K, HumanEval) та оцінювання «LLM-як-суддя», коли інша потужна модель виставляє бали за заданими критеріями.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

У тренувальних даних код і JSON майже завжди супроводжуються markdown-блоками ```json, тому ця звичка глибоко закріплена в патернах генерації і часто «перемагає» коротку пряму інструкцію в промпті.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Тому що частка україномовних текстів у тренувальному корпусі значно менша за англословну, а мала кількість параметрів обмежує здатність моделі якісно засвоїти рідкісніші мовні патерни.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску? Через недетермінізм семплінгу (temperature > 0 означає випадковий вибір серед імовірних токенів) та особливості обчислень з плаваючою комою при паралельному виконанні на GPU.

5. Що таке галюцинація мовної моделі і чому модель радше вигидає факт, ніж визнає незнання?

Галюцинація — це впевнено сформульована, але фактично невірна відповідь. Модель генерує найбільш ймовірний з погляду статистики текст, а не звіряється з базою фактів; оскільки в тренувальних даних

приклади «чесного визнання незнання» трапляються рідко, моделі «вигідніше» видати правдоподібну відповідь, ніж відмовитися відповідати.

Крок 7. Приклад провалу

Запит: «25 студентів, 12 знають Python, 9 знають Java, 4 знають обидві мови. Скільки студентів не знають жодної з мов? Покажи хід розв'язання.» (l1ama3.2:3b, завдання 5, варіант 1).

Відповідь моделі: модель порахувала $25 - 12 - 9 = 4$ і назвала цю різницю фінальною відповіддю, не врахувавши перетин множин (4 студенти, які знають обидві мови, віднялись двічі).

Правильна відповідь: $25 - (12 + 9 - 4) = 25 - 17 = 8$ студентів не знають жодної мови.

Пояснення: модель обрала першу інтуїтивно очевидну арифметичну дію (просте віднімання) замість того, щоб застосувати принцип включення-виключення для перетинних множин. Це типовий приклад того, що правдоподібний на вигляд «хід розв'язання» не гарантує правильності — модель не «розуміє» задачу на комбінаторному рівні, а генерує найбільш звичний патерн обчислення.

Висновок за завданням 5

Оцінювання за фіксованою рубрикою виявилось показовим і відтворюваним способом порівняння моделей: воно чітко розвело малу інструктивну модель (5/12) і міркувальну та хмарну моделі (по 12/12), а подвійний прогін кожного завдання дав змогу відрізнити стабільну поведінку від випадкової.

Найпоказовішими виявились завдання 5 (логіка) і 6 (українська мова) — саме на них мала модель провалювалась найгірше, тоді як формальне завдання 2 (JSON) успішно виконали всі моделі. Для роботи з українськомовним текстом обрано b7qwen3:4b як безкоштовний варіант, здатний працювати локально без втрати якості порівняно з хмарною моделлю.

Завдання 6. Керування випадковістю: temperature і top_p

Виконано через sampling.py.

Таблиця 16 — Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0/5
temperature	0.7	5	—
temperature	1.5	5	0/5
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17 — Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так/ні)	так

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: $temperature = 0,0$ (або близько до нуля). Це підтверджує експеримент: при $0,0$ усі 10 запусків фактичного питання дали однакову відповідь, тобто максимальна передбачуваність — саме те, що потрібно для стабільного парсингу структурованих даних.

2. Яке — генератору варіантів заголовків?

Відповідь: $temperature$ приблизно $0,7-1,0$. За таблицею 16, вже при $0,7$ п'ять запусків творчого завдання дали 5 унікальних назв — достатньо різноманітності без надмірного хаосу, який виникає при $1,5$.

3. Чим вплив top_p відрізняється від впливу $temperature$ за характером, а не за силою?

Відповідь: $temperature$ змінює форму (контрастність) розподілу ймовірностей — «розмиває» або «загострює» його для всіх кандидатів одночасно. top_p , навпаки, не чіпає самі ймовірності, а відсікає «хвіст» — прибирає з розгляду малоїмовірних кандидатів, залишаючи лише мінімальний набір, що в сумі дає задану частку ймовірності.

4. Чи означає $temperature=0$ повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Практично — так: усі 10 запусків фактичного питання при $temperature=0,0$ дали ідентичну відповідь (1 унікальна з 10). Проте теоретичної стовідсоткової гарантії немає — на дуже великому числі повторів чи іншому апаратному забезпеченні можливі розбіжності через особливості обчислень з плаваючою комою й паралелізацію.

5. Наведіть найбезглуздішу відповідь, яку модель видала при $temperature=1,5$, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: При $temperature=1,5$ на творчому завданні (назва стартапу в галузі логістики) модель одного разу запропонувала назву, що поєднувала явно не пов'язані з темою слова. Це сталося тому, що висока температура сильно «сплющує» розподіл ймовірностей, і токени з дуже низькою природною ймовірністю отримують реальний шанс бути обраними — модель фактично випадково блукає серед малоїмовірних продовжень.

1. Що саме математично робить параметр $temperature$ із розподілом ймовірностей наступного токена?

$Temperature$ ділить логіти (сирі оцінки) на своє значення перед застосуванням softmax: при $T < 1$ різниці між логітами посилюються (розподіл загострюється до найімовірнішого токена), при $T > 1$ — згладжуються (розподіл наближається до рівномірного).

2. Чим top_p відрізняється від $temperature$ — обидва впливають на різноманітність, але за яким механізмом?

top_p (nucleus sampling) динамічно обирає найменший набір токенів, чия сумарна ймовірність досягає порогу p , і семплує лише серед них, тоді як $temperature$ не змінює склад кандидатів, а лише форму розподілу ймовірностей серед усіх токенів.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Технічно можна змінювати обидва, але їхній спільний ефект складно передбачити й налаштувати; на практиці зазвичай фіксують один параметр (найчастіше top_p близько $1,0$) і підбирають лише $temperature$, щоб зберегти контрольованість експерименту.

4. Чому навіть при $temperature=0$ мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Через недетермінізм обчислень з плаваючою комою при паралельному виконанні на GPU (порядок підсумовування операцій може відрізнятися між запусками) та можливі відмінності в батчуванні запитів на сервері.

5. Які значення $temperature$ використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Для витягу структурованих даних (JSON, класифікація, факти) типово використовують $temperature$ в діапазоні $0-0,3$, а для творчого письма чи генерації варіантів — $0,7-1,2$.

Висновок за завданням 6

Експеримент підтвердив, що $temperature$ і top_p — це регулятори випадковості вибору токена, а не «розумності» моделі, і працюють вони по-різному: $temperature$ змінює форму всього розподілу ймовірностей, а top_p відсікає малоїмовірний «хвіст» кандидатів. На творчих завданнях підвищення обох параметрів дає бажане розмаїття відповідей (з 1 до 5 унікальних із 5), але на фактичних питаннях та сама випадковість шкодить — при $temperature=1,5$ модель почала «вигадувати» неправильні роки замість стабільної правильної відповіді при $temperature=0$. Для наступних лабораторних, де модель повертає структуровані дані, буде обрано $temperature$ близько 0 — саме це і показав експеримент із 10 повторів, де відповідь жодного разу не відхилилась від очікуваної.

Підсумкове порівняння моделей

Критерій	Локальна мала (3В)	Локальна більша (4В/8В)	Хмарна
Швидкість генерації	~14–108 симв./с (в середньому ~67)	~29 токенів/с* (окремо не бенчмаркалась символами)	~12–237 симв./с (в середньому ~108)
TTFT	~2.0–2.1 с (майже не залежить від довжини)	дані відсутні (не заміряно)	2.8–8.5 с (зростає з довжиною)
Якість, балів /12	5	12	12
Стабільність формату	стабільно (2/2, JSON валідний)	стабільно (2/2, JSON валідний)	стабільно (2/2, JSON валідний)
Потрібні ресурси	~2–3 ГБ RAM, CPU	~4–6 ГБ RAM, CPU/GPU	немає локальних вимог, лише інтернет
Вартість	безкоштовно (крім електрики)	безкоштовно (крім електрики)	безкоштовний ліміт швидко вичерпується (429 Too Many Requests), далі платно
Куди йдуть дані	залишаються на комп'ютері	залишаються на комп'ютері	надсилаються на сервери Google
Працює без інтернету	так	так	ні
Коли обирати саме її	приватні/чутливі дані, прості задачі, офлайн-режим	коли потрібна трохи краща якість за ту саму ціну (0 грн) і є запас RAM	коли критична якість відповіді і швидкість генерації, приватність не критична

Загальні висновки

- Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми — це основа стратегії резервування провайдерів у реальних проєктах;
- Затримка відповіді складається з двох незалежних показників — TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;
- Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;
- Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію можуть проігнорувати моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;
- Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення.